Programming Assignment Report

Design and Analysis of Algorithms

Name: Sony Gwala
Registration Number: 2411021060390
Problem Name: Apple and Orange

1. Problem Statement

Sam's house has an apple tree and an orange tree that produces fruits. The house lies between two points s and t on a number line. The apple tree is located at position, and the orange tree is located at position b. When fruits fall from the trees, they land at a certain distance from their respective tree positions. A negative distance means the fruit falls to the left side of the tree, while a positive distance means it falls to the right side.

The objective is to determine how many apples and oranges land at Sam's house. Some fruit is considered to have landed on the house if its final position lies within the inclusive range [s, t].

2. HackerRank Problem Link

https://www.hackerrank.com/challenges/apple-and-orange

3. Github Submission Link

https://github.com/Tara-86/DAA-Hackerrank-Assignment

4. Algorithm

- Start the program.
- Input s and t represent the starting and ending position of Sam’s house.
- Input a and b representing the position of the apple tree and orange tree.
- Input m and n representing the number of apples and oranges.
- Read the distances of apples from the apple tree.
- Read the distances of oranges from the orange tree.

- For each apple calculate landing position = a + distance.
- If landing position lies between s and t increase apple count.
- For each orange calculate landing position = b + distance.
- If landing position lies between s and t increase orange count.
- Print the number of apples landing on the house.
- Print the number of oranges landing on the house.
- Stop the program.

5. Example Explanation

Suppose Sam's house is between 7 and 11. The apple tree is at position 5 and the orange tree is at position 15. If apples fall at distances [-2, 2, 1], their landing positions are 3, 7, and 6 respectively. Only the apple landing at position 7 lies within the house range.

If oranges fall at distances [5, -6], their landing positions are 20 and 9. Only the orange landing at position 9 lies within the house range. Therefore, the output is 1 apple and 1 orange falling on Sam's house.

6. Complete Java Code

```
import java.util.*;

public class Solution {

    public static void countApplesAndOranges(int s, int t,
int a, int b, int[] apples, int[] oranges) {

        int appleCount = 0;
        int orangeCount = 0;

        for (int apple : apples) {
            int position = a + apple;
            if (position >= s && position <= t) {
                appleCount++;
            }
        }

        for (int orange : oranges) {
            int position = b + orange;
```

```
            if (position >= s && position <= t) {
                orangeCount++;
            }
        }

        System.out.println(appleCount);
        System.out.println(orangeCount);
    }

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int s = sc.nextInt();
        int t = sc.nextInt();

        int a = sc.nextInt();
        int b = sc.nextInt();

        int m = sc.nextInt();
        int n = sc.nextInt();

        int[] apples = new int[m];
        int[] oranges = new int[n];

        for (int i = 0; i < m; i++) {
            apples[i] = sc.nextInt();
        }

        for (int i = 0; i < n; i++) {
            oranges[i] = sc.nextInt();
        }

        countApplesAndOranges(s, t, a, b, apples, oranges);

        sc.close();
    }
}
```

7. Observation / Solution

- The program calculates the landing position of each fruit.
- Apples land at a + distance and oranges land at b + distance.
- The landing position is checked to see if it lies between s and t.
- If the position lies within this range, the fruit is counted as landing on Sam's house.
- The algorithm counts apples and oranges separately.
- Finally, the program prints the total number of apples and oranges that land on the house.

8. Time Complexity

The algorithm processes each apple and orange exactly once. If there are m apples and n oranges, the time complexity is O(m + n). This means the running time increases linearly with the number of fruits.